

Practical No. 8

SQLite CRUD – Contact App

Aim

To develop a simple Android application using **SQLite database** to store, retrieve, update, and delete contact information (name and phone number) and display the contacts using a ListView.

Software Requirements

- Android Studio
 - Kotlin
 - Android SDK
-

Step 1: Create a New Android Studio Project

- Language: Kotlin
 - Template: Empty Views Activity
 - Min SDK: API 21 (or default)
 - File Name: contactapp
-



Step 2: Layout (activity_main.xml)

Very basic UI:

- One EditText (Name)
- One EditText (Phone)
- One Button (Add)
- One Button (View)
- One Button (Update)
- One Button (Delete)
- One ListView

```

<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="vertical"
    android:padding="10dp"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <EditText
        android:id="@+id/name"
        android:hint="Enter Name"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"/>

    <EditText
        android:id="@+id/phone"
        android:hint="Enter Phone"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"/>

    <Button
        android:id="@+id/add"
        android:text="Add"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"/>

    <Button
        android:id="@+id/view"
        android:text="View"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"/>

    <Button
        android:id="@+id/update"
        android:text="Update"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"/>

    <Button
        android:id="@+id/delete"
        android:text="Delete"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"/>

    <ListView
        android:id="@+id/listView"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"/>

</LinearLayout>

```

Step 3: Kotlin Code (MainActivity.kt)

This is the entire logic. Simple and direct.

```

package com.example.contactapp

import android.database.sqlite.SQLiteDatabase
import android.os.Bundle

```

```

import android.widget.*
import androidx.appcompat.app.AppCompatActivity

class MainActivity : AppCompatActivity() {

    lateinit var db: SQLiteDatabase
    lateinit var listView: ListView

    var selectedId = -1

    override fun onCreate(savedInstanceState: Bundle?) {

        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        val name = findViewById<EditText>(R.id.name)
        val phone = findViewById<EditText>(R.id.phone)

        val add = findViewById<Button>(R.id.add)
        val view = findViewById<Button>(R.id.view)
        val update = findViewById<Button>(R.id.update)
        val delete = findViewById<Button>(R.id.delete)

        listView = findViewById(R.id.listView)

        // create database
        db = openOrCreateDatabase("ContactsDB", MODE_PRIVATE, null)

        // create table
        db.execSQL(
            "CREATE TABLE IF NOT EXISTS Contacts(" +
                "id INTEGER PRIMARY KEY AUTOINCREMENT, " +
                "name TEXT, phone TEXT)"
        )

        // ADD
        add.setOnClickListener {

            db.execSQL(
                "INSERT INTO Contacts(name, phone) "
                + "VALUES('${name.text}', '${phone.text}')"
            )

            Toast.makeText(this, "Contact Added",
                Toast.LENGTH_SHORT).show()
        }

        // VIEW
        view.setOnClickListener {

            val cursor = db.rawQuery("SELECT * FROM Contacts", null)

            val list = ArrayList<String>()
            val ids = ArrayList<Int>()

            while (cursor.moveToNext()) {

                val id = cursor.getInt(0)
                val nameValue = cursor.getString(1)
                val phoneValue = cursor.getString(2)
            }
        }
    }
}

```

```

        ids.add(id)
        list.add("$nameValue - $phoneValue")
    }

    cursor.close()

    val adapter = ArrayAdapter(
        this,
        android.R.layout.simple_list_item_1,
        list
    )

    listView.adapter = adapter

    // click contact
    listView.setOnItemClickListener { _, _, position, _ ->

        selectedId = ids[position]

        val parts = list[position].split(" - ")

        name.setText(parts[0])
        phone.setText(parts[1])

        Toast.makeText(this, "Contact Selected",
            Toast.LENGTH_SHORT).show()
    }

    // UPDATE
    update.setOnClickListener {

        if (selectedId != -1) {

            db.execSQL(
                "UPDATE Contacts SET name='${name.text}',
                phone='${phone.text}' WHERE id=$selectedId"
            )

            Toast.makeText(this, "Contact Updated",
                Toast.LENGTH_SHORT).show()

        } else {

            Toast.makeText(this, "Select contact first",
                Toast.LENGTH_SHORT).show()
        }
    }

    // DELETE
    delete.setOnClickListener {

        if (selectedId != -1) {

            db.execSQL(
                "DELETE FROM Contacts WHERE id=$selectedId"
            )

            Toast.makeText(this, "Contact Deleted",
                Toast.LENGTH_SHORT).show()
        }
    }

```

```
        } else {  
            Toast.makeText(this, "Select contact first",  
                Toast.LENGTH_SHORT).show()  
        }  
    }  
}
```

Output

- User enters Name and Phone Number
- Click Add
- Contact is saved in SQLite database
- Click View
- Saved contacts are displayed in ListView
- User selects a contact from ListView
- Selected contact details appear in input fields
- Click Update
- Selected contact is updated in SQLite database
- Click Delete
- Selected contact is deleted from SQLite database
- ListView displays the updated contact list accordingly

